Operating-system structures solutions

The conditions and intermediate states are part of each answer.

OS02-E01 API and system call

1. Controlled syscall entry, argument validation, object-authority checks, protected queues, and required scheduler wakeups execute in the kernel. Library formatting can remain in user mode.
2. No. Success may mean only copying to kernel memory or a device queue. Durability needs a specified write-through, flush, or transaction-commit promise and compliant storage confirmation.
3. Check pointer range, readability, length multiplication overflow, and object lifetime. Copy to kernel-owned storage and use that copy to prevent subsequent user mutation.

Mode, durability, and boundary validation are separate answers.

OS02-E02 Microkernel trade-offs

1. A service in its own domain need not be able to overwrite kernel memory. Smaller authority can limit damage and make local restart easier.
2. Cross-domain IPC adds context switches/copies. Service managers, IPC parsers, and capability transfer can become new privileged attack surfaces. Performance pressure can encourage excessive sharing.
3. Minimize each service's authority with capabilities. Fuzz IPC parsing and validate authority reduction and crash recovery end to end. Architecture names alone do not prove these boundaries correct.

Explain both isolation and IPC boundaries.

OS02-E03 Boot trust chain

1. No. An untrusted loader can change the image, skip verification, falsify the result, or supply malicious configuration.
2. An immutable trust root must authorize/verify the next stage, and each trusted predecessor must verify the following image and enforce rejection. Logging alone is insufficient.
3. Restrict accepted signers, revocations, versions, and policy; prevent runtime bypass of verification. A signature establishes authorized provenance/integrity, not absence of bugs.

Every link must hold.

Operating-system structures solutions (continued)

OS02-E04 Short writes and copy invariant

1. Read 4096 gives source 4096, destination 0. Write 1000 gives destination 1000, pending3096; write 3096 gives4096. Read 1904 gives source 6000; write 1024 gives destination 5120, pending880; write 880 gives6000. Read 0 gives EOF. Positive read total4096+1904=6000; write total1000+3096+1024+880=6000.
2. First chunk loses3096; second loses880. Total3976 lost and2024 written. EOF cannot recover a buffer already overwritten.
3. W≤R, and buffer[j: n] is exactly the contiguous unread-to-destination source suffix. Positive k increments W, j without duplicating the completed prefix. Read again only at j=n. Zero-progress EINTR retries the same remainder; positive returns advance first. Handle write 0 without an endless loop. EOF with an empty pending interval proves W=R.
4. No. File fsync under the storage contract covers data/required metadata; directory fsync separately covers the new name. Direct copy exposes partial destination state. Atomic replacement requires temporary file, sync, rename, and directory sync protocol.

Match bytes, pending intervals, and persistence objects.

OS02-E05 Pipeline references and EOF

1. Cat keeps fd1→writer and closes3/original4. Wc keeps fd0→reader and closes original3/4. Shell closes3/4. Duplicated references count even when their descriptor numbers differ from the original pipe fds.
2. Shell or wc can retain a write-end reference. An empty pipe with a live writer can block awaiting future bytes. Shell waiting for wc while retaining its writer causes an unnecessary stalemate.
3. Yes. Cat writes nothing and closes the writer. Wc correctly counts an empty stream and exits0. Whole-pipeline status is not the same as every-stage success; collect both wait statuses.
4. No. Two forks create two children. Exec replaces their images while preserving process identity. The shell still waits without execing itself.

EOF depends on references, not process names.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Operating-system structures solutions (continued)

OS02-E06 ABI range and TOCTOU

1. 24+8=32 B only in this memory-block ABI. API specifies behavior/types; ABI machine layout/entry/return; implementation resolves paths, rights, and descriptors. This is not a universal Linux-open copy count.
2. Require p<U and n≤U−p. U−p=0x100. Length0x100 is valid with exclusive end0xC0000000/final byte0xBFFFFFFF. Length0x101 crosses the boundary. Mapping/rights/fault-safe access remain separate.
3. Mathematical product0x100000004 truncates to4. Require count≤SIZE_MAX/4=0x3FFFFFFF first, so reject it. Do not allocate4 B and iterate the original count.
4. Another thread can change mappings or arguments, and actual access can fault. Use fault-aware copying, bounded private storage, and internal-format checks. Namespace stability additionally needs kernel object references/synchronization.

Numerical range does not establish successful access.

OS02-E07 Policy and layer dependency

1. q 1 gives A0–1 B1–2 C2–3 A3–4 B4–5 A5–6 B6–7 A7–8 A8–9; completion 9,7,3, response0,1,2. q 4 gives A0–4 B4–7 C7–8 A8–9; completion 9,7,8, response0,4,7.
2. Means1 ms and11/3 ms. Adjacent different tasks produce7 versus3 switches; final A→A is not a task switch. Nonzero switching costs require new wall-time/response calculations.
3. Timer, context save/restore, and controlled ready queues are mechanisms. RR, quantum, and queue order are policy. Timer precision/nonpreemptible intervals constrain them.
4. Strict descending dependencies imply l(pager)>l(driver)>l(scheduler)>l(pager), a contradiction. Keep scheduler metadata and fault/I/O-progress resources resident, and make lower scheduler primitives independent of paging. Then layer other work through established contracts.

Give the schedule and the general cycle contradiction.

Operating-system structures solutions (continued)

OS02-E08 Trace, tail latency, and race

1. CPU[0,1)+[9,10)=2; blocked[1,8)=7; ready[8,9)=1; total10 ms. Completion8 is neither return nor immediate running.
2. Sum107, mean13.375 ms. ceil(.99×8)=8, p99=100 ms. This small sample statistic does not establish a stable population quantile.
3. T1r0, T2r0, T2w1, T1w1 produces1. Printf can change scheduling into serial RMWs producing2. Observation changes timing without synchronization; nonreproduction does not remove the race.
4. No. Both can read 0 before either acquires the write lock, then serialize writes1. Protect the full RMW with one mutex or suitable atomic RMW. The invariant must exclude every allowed interleaving, not only the exposed trace.

CPU sampling and blocked latency measure different things.

OS02-E09 Root closure and trust chain

1. No. Reading S's module needs root, while reading root needs S. Build it in or include it and its dependencies in readable initramfs.
2. Closure proves components can be obtained in order. ABI compatibility, working hardware, correct driver logic, root identity, and valid init/interpreter are still needed.
3. No. The loader can bypass checks or change results/images. A trusted predecessor must authorize/verify it and enforce policy. Specify coverage of kernel, initramfs, and configuration.
4. Measurement records content without necessarily rejecting it. Signature validation establishes authorized provenance/integrity without proving bug freedom. Anti-rollback/revocation rejects otherwise validly signed old versions. None establishes CPU/driver ABI compatibility.

Dependency reachability, behavioral correctness, and provenance need separate evidence.

Operating-system structures solutions (continued)

Verification code appendix: system_boundary_models.py

```
"""Chapter 2: self-contained teaching models; not an OS/kernel emulator."""
from collections import deque


def copy_chunks(data, read_sizes, write_limits):
    """Preserve order under short reads/writes; each limit must make progress."""
    destination = bytearray()
    source_offset = 0
    writes = iter(write_limits)
    trace = []
    for requested in read_sizes:
        if requested <= 0:
            raise ValueError("positive read request required")
        chunk = data[source_offset:source_offset + requested]
        source_offset += len(chunk)
        trace.append(("read", len(chunk), source_offset, len(destination)))
        if not chunk:
            break
        consumed = 0
        while consumed < len(chunk):
            limit = next(writes)
            if limit <= 0:
                raise ValueError("write made no progress")
            count = min(limit, len(chunk) - consumed)
            destination.extend(chunk[consumed:consumed + count])
            consumed += count
            trace.append(("write", count, source_offset, len(destination)))
    return bytes(destination), trace


def user_range(pointer, length, upper=0xC0000000):
    """Only nonempty numeric ranges; mappings/faults are separate concerns."""
    return 0 <= pointer < upper and 0 < length <= upper - pointer


def allocation_bytes(count, element_size, word_bits=32):
    maximum = (1 << word_bits) - 1
    if count < 0 or element_size <= 0 or count > maximum // element_size:
        raise ValueError("allocation size overflows")
    return count * element_size


def round_robin(bursts, quantum):
    """All arrive at zero; zero switch cost; order is dictionary insertion order."""
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Operating-system structures solutions (continued)

```
    if quantum <= 0 or any(value <= 0 for value in bursts.values()):
        raise ValueError("positive bursts/quantum required")
    remaining = dict(bursts)
    queue = deque(bursts)
    time = 0
    trace, first, completed = [], {}, {}
    while queue:
        name = queue.popleft()
        first.setdefault(name, time)
        duration = min(quantum, remaining[name])
        trace.append((time, time + duration, name))
        remaining[name] -= duration
        time += duration
        if remaining[name]:
            queue.append(name)
        else:
            completed[name] = time
    return {"trace": trace, "response": first, "completion": completed,
            "switches": sum(a[2] != b[2] for a, b in zip(trace, trace[1:]))}


def dependency_closure(available, requirements):
    available = set(available)
    while True:
        added = {name for name, needs in requirements.items()
                 if set(needs) <= available} - available
        if not added:
            return available
        available.update(added)


if __name__ == "__main__":
    source = bytes(index % 251 for index in range(6000))
    result, trace = copy_chunks(source, [4096, 4096, 4096], [1000, 3096, 1024, 880])
    assert result == source
    print("copy:", trace)
    for quantum in (1, 4):
        print("RR", quantum, round_robin({"A": 5, "B": 3, "C": 1}, quantum))
    print("numeric range:", user_range(0xBFFFFF00, 0x100), user_range(0xBFFFFF00, 0x101))
    dependencies = {"S": {"initramfs"}, "F": {"initramfs"}, "mount-root": {"S", "F"}}
    print("early dependencies:", sorted(dependency_closure({"K", "initramfs"},
dependencies)))
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study